

Programación II

Desarrollo en Java

Clase N° 17:

Manejo de archivos de texto

Profesora Carolina Archuby



TEMAS A DESARROLLAR

- Concepto de archivo
- Archivos en Java
- Diferencia entre archivos de texto y archivos binarios.
- La clase File.
- Operaciones con archivos de texto (crear, leer, escribir, borrar, cerrar)

Concepto de Archivos

=> Un archivo (o fichero) es un conjunto de bits almacenados en un dispositivo, y accesible a través de un camino de acceso (pathname) que lo identifica.

=> ¿Por qué usar archivos? Se usan cuando tenemos que guardar los datos de nuestro programa para poder recuperarlos más adelante y el volumen de datos no es muy elevado y no justifica el uso de bases de datos.

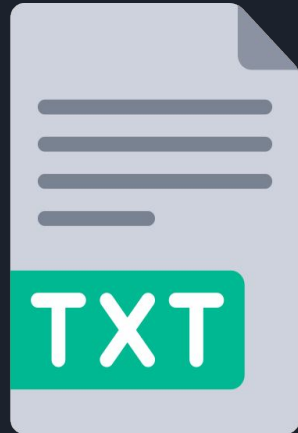
Archivos

=> Suelen organizarse en estructuras jerárquicas de directorios. Estos directorios son contenedores de ficheros (y de otros directorios) que permiten organizar los datos del disco.

=> El nombre de un archivo está relacionado con su posición en el árbol de directorios que lo contiene, lo que permite no sólo identificar unívocamente cada fichero, sino encontrarlo en el disco a partir de su nombre.

Archivos de texto

- => Los datos se almacenan como caracteres **ASCII** o **Unicode**.
- => Son archivos que podremos **crear** desde un programa Java y **leer** con **cualquier editor de texto**, o bien crear con un editor de textos y leer desde un programa Java.
- => Los archivos de texto son adecuados para almacenar y procesar datos que se pueden **leer y editar fácilmente**.



Archivos binarios

=> Los datos se almacenan en su forma binaria original (como una **secuencia de bits**)

=> Son **más eficientes** en el almacenamiento y procesamiento de datos que los archivos de texto.

=> Son adecuados para almacenar y procesar **datos que no están diseñados para ser legibles por humanos**, como imágenes, audio, video y cualquier otro tipo de archivo que requiera ser procesado por un programa.



Operaciones con archivos de texto

Para manipular archivos, siempre tendremos los mismos pasos:

- 1) **Abrir** el archivo → Si no abrimos el archivo, obtendremos un mensaje de error al intentar acceder a su contenido.
- 2) **Escribir** datos o **leerlos**.
- 3) **Cerrar** el archivo → Si no cerramos el archivo, puede que realmente no se llegue a guardar ningún dato

Clases propias de Java a utilizar para ESCRIBIR Archivos de texto

=> **File**: El objeto de la clase File contiene el nombre del archivo, la ruta y más propiedades relativas a él. Esta clase define métodos para conocer propiedades del archivo, como la última modificación, permisos de acceso, tamaño, etc.

=> **FileWriter** : Es una clase que permite escribir caracteres en un archivo de texto con **métodos básicos, como write()**, sin funcionalidades avanzadas para formatear la salida. Los datos se escriben directamente en el archivo sin almacenarse temporalmente en un buffer. Para mejorar el rendimiento suele combinarse con un BufferedWriter. Lanza excepciones como IOException directamente cuando ocurre un error de lectura.

=> **PrintWriter**: Es una clase que permite escribir texto en un archivo de texto y proporciona **métodos más avanzados, como print() y println() para escribir datos formateados**. Suele combinarse con un BufferedWriter. No lanza excepciones cuando ocurre un error de lectura. En su lugar, se puede verificar si hubo error usando el método checkError().

Clases propias de Java a utilizar para LEER Archivos de texto

=> FileReader : Es la **clase básica** que se usa para leer caracteres directamente de un archivo de texto. **No usa buffer** y sólo tiene métodos básicos, como read(), que permite leer un carácter o un array de caracteres a la vez.

=> BufferedReader : Es una **clase más avanzada** que se usa para leer texto de un archivo de texto. Usa un **buffer** interno y proporciona métodos más convenientes para leer líneas enteras de texto, como readLine(), que permite leer una línea completa de texto de una sola vez.

=> El uso de buffer reduce la cantidad de veces que se accede al disco, **mejorando el rendimiento**, y es más adecuado cuando se trabaja con **archivos grandes**.

Crear archivo

```
public static void crearArchivo(String nombreArchivo) {  
    File file = new File(nombreArchivo);  
    try {  
        PrintWriter salida = new PrintWriter(file);  
        //0: FileWriter fileWriter = new FileWriter(nombreArchivo); //sin usar File  
        ...  
        salida.close();  
    } catch (FileNotFoundException e) {  
        e.printStackTrace();  
    }  
    System.out.println("El archivo se ha creado correctamente");  
}
```

Escribir archivo (sobrescribir)

```
public static void sobrescribirArchivo(String nombreArchivo, String contenido){  
    File file = new File(nombreArchivo);  
    try {  
        PrintWriter salida = new PrintWriter(file);  
        salida.println(contenido);  
        // o: FileWriter salida = new FileWriter(nombreArchivo); // sin usar File  
        // y: salida.write(contenido);  
        salida.close(); // igual si usamos FileWriter  
        System.out.println("Archivo escrito correctamente");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```

Escribir archivo (agregar información)

```
public static void agregarArchivo(String nombreArchivo, String contenido){  
    File file = new File(nombreArchivo);  
    try {  
        PrintWriter salida = new PrintWriter(new FileWriter(file, append: true));  
        salida.println(contenido);  
        //o: FileWriter salida= new FileWriter(nombreArchivo, true); //sin usar File  
        // y: salida.write (contenido) :  
        salida.close(); // igual si usamos FileWriter  
        System.out.println("Datos agregados correctamente al archivo");  
    } catch (IOException ex) {  
        ex.printStackTrace();  
    }  
}
```

Leer Archivo

```
public static void leerArchivo (String nombreArchivo) {
    File file = new File(nombreArchivo);
    try {
        BufferedReader entrada= new BufferedReader(new FileReader(file));
        // o: FileReader entrada= new FileReader(nombreArchivo); //sin File
        String lineaActual = null;
        // o: int caracterInt; (porque read lee char por char y retorna su valor int
        while ( (lineaActual= entrada.readLine()) != null) {
            // o: while ((caracterInt= entrada.read()) != -1) {
                System.out.println(lineaActual);
                // o: System.out.print((char) caracterInt);
            }
        }
        entrada.close();
    }catch(FileNotFoundException e){
        e.printStackTrace();
    }
}
```

Borrar todo el contenido del archivo

```
public static void borrarContenidoArchivo (String nombreArchivo) {  
    try {  
        FileWriter salida = new FileWriter(nombreArchivo, append: false);  
        salida.write( str: "");  
        salida.close();  
        System.out.println("El archivo se ha borrado correctamente");  
    } catch (IOException e) {  
        e.printStackTrace();  
    }  
}
```


Eliminar Archivo

```
public static void eliminarArchivo (String nombreArchivo) {  
    File file = new File (nombreArchivo) ;  
    try {  
        if (file.delete()) {  
            System.out.println("Archivo eliminado con éxito.");  
        } else {  
            System.out.println("No se pudo eliminar el archivo.");  
        }  
    } catch (Exception e){  
        System.out.println("Ocurrió un error al intentar eliminar el archivo.");  
        e.printStackTrace();  
    }  
}
```